

설비 고장을 예측하는 AI 데이터 분석

고장 임박 분류 모델

랜덤 포레스트 고장 임박 분류

C O N T E N T S

목차

01 RUL 기반 고장 임박 문제

02 라벨 생성 및 데이터 준비

03 랜덤 포레스트 고장 임박 분류

- 모델과 트리 — 결정 트리에서 랜덤 포레스트까지
- 학습과 예측 — fit / predict / predict_proba
- 해석과 확장 — feature importance, MIMII 적용, 전체 파이프라인

01

모델과 트리

결정 트리 직관에서 랜덤 포레스트까지

모델이란 — 입력을 받아 답을 내는 상자

무언가를 넣으면 답이 나오는 상자 — 상자 안의 규칙은 학습으로 채워짐

01 · 입력

상자에 들어가는 값

입력 x
= 센서값

여러 센서값을 한 번에 넣어 모델에게 판단을 요청

02 · 규칙

상자 안에 들어 있는 것

학습으로 채워진
판단 규칙

정답 붙은 데이터를 보며 입력과 정답의 관계를 스스로 익힘

03 · 출력

상자가 내놓는 답

0 또는 1
괜찮음 / 곧 고장

자판기에 동전을 넣으면 음료가 나오듯 센서값을 넣으면 답이 나옴

상자 속 규칙은 학습으로

빈 상자 → 정답 붙은 데이터로 규칙 채움 → 새 입력에 답할 수 있는 상자

BEFORE

학습 전

규칙이 없는 빈 상자

아무것도 익히지 않은 백지 상태

LEARNING

학습

정답 붙은 데이터를 보여 규칙을 채움

입력과 정답의 관계를 스스로 다듬는 단계

AFTER

학습 후

새 입력에 답할 수 있는 상자

예측을 내놓을 준비 완료

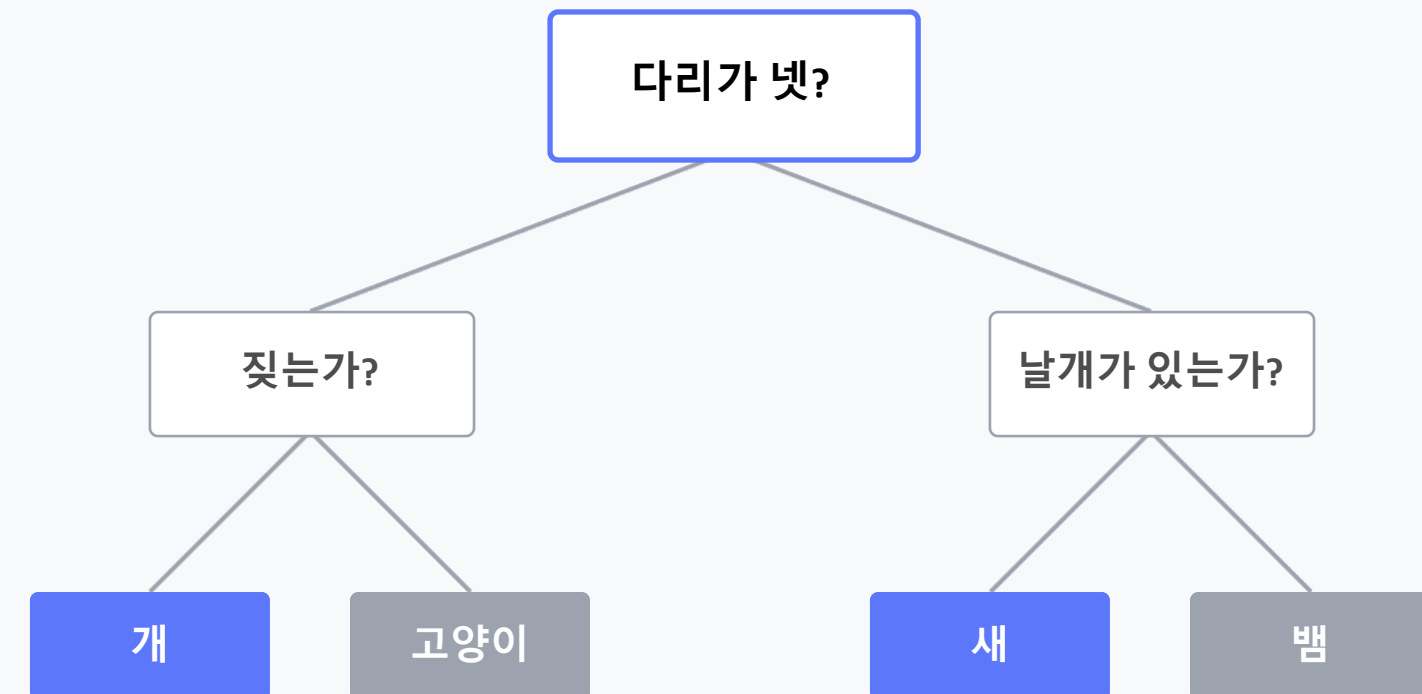
결정 트리 직관 — 스무고개로 판단하기

예/아니오 질문을 차례로 던져 답에 도달 — 가지가 갈라져 나무(트리) 모양

KEY IDEA

예/아니오 질문을 **차례로** 던져
답에 도달하는 방식

- 스무고개 놀이와 똑같은 원리
- 다리가 넷인가, 짚는가 물으며 답을 좁힘
- 질문이 가지처럼 갈라져 나무 모양



결정 트리가 고장을 판단하는 법

센서값에 질문을 던지고 답에 따라 가지를 나눠 0/1에 도달

STEP 1

센서값에 질문

sensor_4가
어떤 값보다 큰가?

입력 x의 한 열을 보고 묻는 일



STEP 2

답에 따라 가지 나눔

예/아니오로 갈라져
두 갈래의 가지로 진행

필요하면 다른 센서로 또 질문



STEP 3

0 또는 1에 도달

질문을 이어가다
곧 고장(1) 또는 괜찮음(0)

가지의 끝 = 트리의 잎(예측값)

결정 트리가 센서값으로 나누는 방식

정답이 가장 깔끔하게 갈라지는 센서와 기준값을 데이터에서 직접 찾음

01 · 분리 기준

정답이 가장
깔끔하게 갈라지는 곳

어느 센서를 어떤 기준값으로 자를지 모든 후보를 비교

02 · 좋은 분리

한쪽엔 1,
다른 쪽엔 0 몰리게

sensor_4가 1400보다 크면 대부분 1, 작으면 대부분 0 같은 기준

03 · 자동

사람 지시 없이
데이터만 보고 진행

어디서 자를지 일일이 알려주지 않아도 스스로 결정

나누고 또 나누기

섞여 있으면 다른 센서로 또 나눠 가지 끝에 깨끗한 묶음만 남김

RECURSIVE SPLIT

한 번 나눈 뒤에도 섞여 있으면
다른 센서로 또 나눔

- 반복하면 가지 끝에 거의 1만 또는 0만 모임
- 복잡한 계산은 scikit-learn이 알아서 처리
- 우리는 큰 그림만 잡으면 충분

BEFORE → AFTER

1차 분리 — 여전히 섞임

1 1 0 1 0 1 1 0

0 1 0 0 1 0 0 0



2차 분리 — 깨끗한 묶음

1 1 1 1

0 0 0 0

1 1 1

0 0 0 0 0

가지 끝에 같은 정답만 모일 때까지 반복

똑똑한 트리 하나면 충분할 것 같은데, 왜 여러 개를 쓸까?

트리 하나의 한계

학습 데이터에 지나치게 맞춰져 새 데이터에 약하고 불안정

LIMIT 01 · 과적합

학습 데이터에
지나치게 맞춤

본 데이터의 결만 따라가 일반화에 약함

LIMIT 02 · 약한 일반화

문제집을 통째로
외운 학생

본 문제는 완벽히 풀지만 살짝 다른 새 문제엔
당황

LIMIT 03 · 불안정

데이터가 조금만 바뀌어도
결과가 크게 흔들림

가지 모양이 획 달라져 안정적인 판단 어려움

약점의 해결책 — 여러 트리

한 학생에게만 의존하지 말고 서로 다르게 공부한 여러 학생의 다수결

SOLUTION

서로 조금씩 다른
여러 트리의 의견을 모음

- 각자 다르게 공부한 여러 트리를 모음
- 한 트리의 치우침이 전체 속에서 없어짐
- 이 발상이 바로 랜덤 포레스트

여러 학생의 다수결

학생 A	학생 B	학생 C	학생 D	학생 E
곧 고장	곧 고장	괜찮음	곧 고장	곧 고장

MAJORITY VOTE

4 : 1 → 최종 결론 1 (곧 고장)

랜덤 포레스트 — 여러 트리의 다수결

트리 수십~수백 그루의 0/1 투표 결과 중 가장 많은 답이 결론

01 · 숲

서로 조금씩 다른
수십~수백 그루

각 트리는 데이터·센서의 일부만 보고 자라 서로 다른 관점을 가짐

포레스트 = 숲

02 · 투표

새 데이터에
모든 트리가 0/1로 투표

트리 100그루가 모두 개별 예측을 내놓는 단계

각 트리는 한 표씩 행사

03 · 다수결

가장 많은 표를 받은
답이 최종 결론

100그루 중 70이 1을 골랐다면 최종 결론은 1

개별 트리의 실수를 열게 만드는 장치

각 트리가 일부만 보고 자라기 때문에 의견이 갈라짐 — 그래서 다수결이 의미를 가짐

다수결이 더 나은 이유

한 명보다 여러 전문가의 의견 — 한 트리의 실수가 다수결 속에 묻힘

01 · 집단지성

한 명보다
여러 전문가

여럿이 의견을 모으면 한 명의 판단보다 신뢰도 상승

02 · 오차 보정

실수를 다른 트리가
바로잡음

한 트리가 잘못 판단해도 나머지가 제대로 판단하면 다수결 속에 묻힘

03 · 안정성

숲은 트리 하나보다
안정적

데이터가 살짝 바뀌어도 결과가 획 흔들리지 않음

더 복잡한 모델도 많은데 왜 하필 랜덤 포레스트가 첫 모델로 적합할까?

랜덤 포레스트가 입문에 적합한 이유

설정 간단, 단위에 강함, 안정적, 해석 가능 — 네 가지 큰 장점

01 · 설정 간단

기본값만으로 충분한 성능

조정할 게 많은 모델과 달리 거의 그대로 써도 좋은 결과

02 · 단위에 강함

센서값 크기가 달라도 작동

별도 단위 맞추기 없이도 잘 작동
, 입문자 부담 감소

03 · 안정적

**다수결로 결과가 흔들리지
않음**

데이터가 약간 바뀌어도 비슷한
결과를 유지

04 · 해석 가능

어떤 센서가 중요했는지 알려줌

feature importance로 판단 근거를
사람이 확인 가능

단위 맞추기가 필요 없는 이유

값의 크기 비교가 아니라 "기준보다 큰지 작은지"로만 나누기 때문

01 · 분할 방식

크기 비교 아닌
기준 초과/미만으로만

sensor_4가 1400보다 큰가? 같은 임계값 질문만 사용

02 · 결과

단위 맞추기 작업 없이
바로 작동

온도·압력·진동 단위가 제각각이어도 별도 보정 없이 학습 가능

03 · 입문자에게

손이 하나
덜 가는 큰 장점

스케일링 단계를 잊어도 모델이 그대로 돌아감

n_estimators(트리 개수) 직관

숲에 심을 트리 개수 — 많을수록 안정적, 단 어느 선부터는 시간만 늘어남

01 · 이름의 뜻

estimator = 트리 하나,
n = 개수

숲에 심을 트리를 몇 그루로 할지 정하는 설정

02 · 많으면

다수결이
안정적

트리가 많을수록 한두 트리의 치우침이 묻혀
결과 변동이 작음

03 · 지나치면

효과는 줄고
시간만 늘어남

어느 선을 넘으면 정확도는 거의 그대로, 학습
시간만 늘어남

우리가 쓸 기본값

입문 단계의 무난한 선택 — `n_estimators = 100`

DEFAULT VALUE

100

그루의 트리로 시작

`n_estimators = 100`

01 · 무난한 시작점

보통 100그루를 기본으로 시작하는 것이 입문 단계의 정석

02 · 안정성과 속도의 균형

다수결의 안정성을 충분히 누리며 학습이 금세 끝남

03 · 우리 수업도 100

개수 고민 없이 본질에 집중 — 샘플 데이터에서 거의 즉시 학습 완료

02

학습과 예측

RandomForestClassifier · fit · predict · predict_proba

RandomForestClassifier 만들기

sklearn.ensemble에서 분류기를 꺼내 트리 100그루로 모델 생성

CODE · MAKE_MODEL.PY

```
# sklearn.ensemble에서 RandomForestClassifier를 꺼낸다
from sklearn.ensemble import RandomForestClassifier

# 트리 100그루, random_state=42로 결과 고정
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

ensemble = 여러 모델을 모음 · random_state로 결과를 고정해 누구나 같은 결과 재현

아직은 빈 상자

모델을 만들었지만 아직 아무것도 배우지 않은 상태 — 다음은 fit

01 · 현재 상태

아무것도 배우지 않은
빈 모델

객체는 만들어졌지만 안에 든 트리 100그루는
아직 백지

02 · 비유

밭은 갈았지만
씨앗은 안 뿌린 상태

숲을 만들 준비만 한 단계 — 아직 어떤 데이터
도 만난 적 없음

03 · 다음 단계

실제 학습은
fit에서 시작

데이터를 보여 주며 진짜 학습을 시키는 단계
가 다음

.fit() — 학습이 실제로 하는 일

학습용 입력과 정답을 fit에 건네 한 줄로 트리 100그룹을 동시에 학습

CODE · FIT.PY

```
# 학습용 입력 X_train과 정답 y_train으로 학습
model.fit(X_train, y_train)
```

NOTICE

fit에는 학습용(train)만 — 평가용(test)은 시험을 위해 절대 넣지 않음

test가 학습에 새면 평가는 의미를 잃음

fit 한 줄이 어떤 센서값일 때 정답이 1이고 0인지를 익히는 단계

fit은 한 번만

딱 한 번만 제대로 — 같은 데이터로 여러 번 fit해도 더 똑똑해지지 않음

01 · fit 동안

트리 100그룹이
동시에 자람

각 트리가 데이터를 보고 가지를 나누며 모델
안에 저장됨

02 · 소요 시간

샘플 데이터는
몇 초면 끝

데이터가 작아 학습이 거의 즉시 완료됨

03 · 재실행

여러 번 fit해도
더 똑똑해지지 않음

같은 데이터로 다시 fit하면 처음부터 다시 학
습되는 것뿐

fit이 끝난 모델 안에는 정확히 무엇이 들어 있을까?

학습이 끝난 모델은 무엇을 갖고 있나

데이터에서 찾아낸 판단 규칙 — 잘 자란 트리 100그루

BEFORE · 학습 전

**백지 노트 —
던질 질문조차 없음**

- 어떤 센서가 중요한지 모름
- 어떤 기준값으로 자를지 모름
- 0인지 1인지 답할 능력 없음



AFTER · 학습 후

**정답 풀이가 빠곡한 노트 —
새 데이터에 답할 준비 완료**

- 각 트리는 어떤 센서·기준으로 나눌지 갓춤
- 트리 100그루의 규칙이 모델 안에 저장
- 데이터에서 찾은 질문들로 무장

좋은 데이터가 좋은 모델을 만든다

모델 속 규칙은 우리가 보여 준 데이터에서 정리한 것

01 · 규칙의 출처

모델 속 규칙은
데이터에서 나옴

트리의 가지는 외부 지식이 아닌 학습 데이터
에서 정리한 결과

02 · 결론

데이터 품질이
모델 품질을 좌우

잘못된 라벨, 누락된 값은 그대로 모델 안에 들
어감

03 · 앞 모듈의 의미

데이터 준비가
중요했던 이유

RUL 라벨링, X/y 분리, train/test 분리에 공을 들
인 까닭

.predict() — 새 데이터에 답하기

아껴 둔 평가용 `x_test`로 각 행마다 0/1 판단을 받음

CODE · PREDICT.PY

```
# 평가용 x_test의 각 행에 대해 0/1 판단을 받음  
y_pred = model.predict(X_test)
```

FIT

오래 걸리는 공부 단계

트리들이 자라며 규칙을 채우는 단계

PREDICT

이미 만든 규칙에 대입 — 거의 즉시

새 입력을 트리에 통과시켜 0/1을 받음

한 번 학습한 모델은 새 입력이 들어올 때마다 빠르게 답을 내놓음

예측할 때 주의할 점

학습 때와 같은 형태로 — 모델은 학습 때 본 형태의 입력만 이해

주의 항목	설명
입력 형태	predict에 넣는 입력 형태는 학습 때와 같아야 함
컬럼 수와 순서	학습에 센서 6개를 썼다면 예측도 같은 6개를 같은 순서로
정답 금지	predict에는 정답(y)을 넣지 않음

predict는 입력 x만 — 정답을 같이 넣는 건 평가 단계의 일

예측 결과의 형태 (0/1 배열)

행마다 하나씩 0 또는 1이 담긴 배열 — 순서는 x_{test} 의 행 순서와 동일

OUTPUT SHAPE

입력 100행이면
답도 100개

- 결과는 0과 1이 죽 늘어선 배열
- 순서가 x_{test} 와 같아 나란히 비교 가능
- 나중에 실제 정답 y_{test} 와 짝지을 수 있음

EXAMPLE

y_{pred} 앞부분



■ 0 — 괜찮음

■ 1 — 곧 고장

첫 시점은 괜찮음, 둘째·셋째는 곧 고장

순서가 보존된다는 의미

예측·실제·설비 번호를 같은 순서로 나란히 짝지을 수 있음

01 · 짝짓기

예측 배열의 세 번째 값
= X_{test} 세 번째 행의 답

행 인덱스가 그대로 유지되므로 입력과 출력
이 1:1로 대응

02 · 설비 번호

설비 번호와도
같은 순서

어느 설비가 곧 고장으로 예측됐는지 정확히
연결 가능

03 · 활용의 바탕

예측 · 실제 · 설비 번호
나란히 비교 가능

이 단순한 사실이 우선순위 목록 만들기의 기반

예측 확률(predict_proba) 직관

100그룹 중 몇 그룹이 1에 투표했는지를 확률로 알려줌

CASE 01 · 강한 확신

[0.10, 0.90]

곧 고장을 강하게 확신

100그룹 중 90그룹이 1에 투표한 결과

CASE 02 · 애매함

[0.45, 0.55]

1이지만 확신은 약함

55그룹만 1에 투표 — 한 번 더 지켜볼 후보

의미

각 답에 대한
확신 정도를 확률로

- [0일 확률, 1일 확률]
- 두 값을 더하면 1
- 0.5에 가까울수록 애매

같은 1이라도 간신히 1인지 확실하게 1인지 알 수 있는 정보

확신도를 현장에 활용하기

단순한 0/1보다 풍부한 판단 재료 — 우선순위를 더 세밀하게

예측	확신도	현장 조치
1·곧 고장	높음 (0.9 이상)	더 시급히 점검 — 가장 위험한 설비로 분류
1·곧 고장	애매함 (0.5 부근)	한 번 더 지켜봄 — 다음 사이클의 관찰 대상
0·괜찮음	높음	정상 운전 계속 — 추가 조치 불필요

아주 확실한 것은 즉시, 애매한 것은 관찰 — 현장 우선순위를 세밀하게 조정

예측을 실제 정답과 눈으로 비교하기

예측과 실제를 같은 순서로 나란히 놓고 한 줄씩 맞춰 보기

가장 직관적인 점검

예측 1에 실제도 1이면 맞음,
예측 1에 실제 0이면 틀림

- 예측 배열과 y_test를 같은 순서로 나란히
- 앞쪽 몇 개만 훑어도 대체로 잘 맞는지 감이 옴
- 정확한 점수 계산은 다음 파트의 평가 지표에서

EXAMPLE

행	예측	실제	결과
			맞음
			맞음
			틀림
			맞음
			맞음

5행 중 4행 맞음 — 대체로 잘 맞는 감

눈비교에서 보이는 네 경우

예측 x 실제 조합 — 실제 1, 예측 0의 위험 놓침이 가장 치명적

CASE 01 · 정답 예측 1 실제 1 곧 고장을 제대로 맞힘 진짜 위험을 잡아낸 이상적 케이스	CASE 02 · 정답 예측 0 실제 0 정상을 제대로 맞힘 멀쩡한 설비는 그대로 통과시킴	CASE 03 · 치명적 오답 예측 0 실제 1 위험을 놓침 (더 치명적) 고장이 다가오는데 놓침 — 가장 피해야 할 케이스	CASE 04 · 가벼운 오답 예측 1 실제 0 괜한 점검을 부름 헛걸음의 비용 — 위험 놓침보다는 덜 치명적
--	--	---	---

같은 오답이라도 가중치가 다름 — 위험 놓침이 헛걸음보다 훨씬 무겁다

모델은 본 적 없는 데이터에 약하다

예측은 진리가 아니라 근거 있는 추정 — 학습 데이터와 다른 설비엔 약함

01 · 강점

학습 때 본 것과
비슷한 데이터는 잘 맞힘

데이터 분포가 같으면 트리들이 익힌 기준이
그대로 통함

02 · 약점

전혀 다른 종류의
새 데이터엔 약함

특정 엔진으로 배운 모델을 다른 설비에 그대로 쓰면 엉뚱한 답

03 · 자세

예측은 진리가 아니라
근거 있는 추정

새 현장엔 비슷한 데이터로 다시 학습시키는
게 안전

사람과 함께 쓰는 도구

모델은 사람을 대체하지 않고 돕는 도구 — 경험과 함께 쓰는 태도가 중요

01 · 의심하는 자세

**예측이 이상하면
사람이 직접 확인**

모델이 늘 옳다는 가정 대신 검증 가능한 출발
점으로 활용

02 · 적용 범위

**성격 다른 설비엔
특히 조심**

학습 데이터와 분포가 다른 설비엔 결과를 그
대로 믿지 않기

03 · 협업

**사람을 대체하지 않고
돕는 도구**

사람의 경험과 함께 쓰는 태도가 가장 안전한
사용법

어떤 센서가 판단에 크게 쓰였는지 알 수 있다면 결과를 더 깊이 이해 가능

03

해석과 확장

feature importance · MIMII 적용 · 전체 파이프라인

어떤 센서가 판단에 크게 쓰였나

feature_importances_ — 각 센서가 판단에 쓰인 정도를 점수로 표시

01 · 무엇인가

각 센서가 판단에
쓰인 정도

학습된 모델의 feature_importances_로 확인 가능

02 · 높은 점수의 의미

자주, 효과적으로
가지를 나눈 센서

정답을 가르는 데 큰 역할을 할수록 높은 점수

03 · 점수 합계

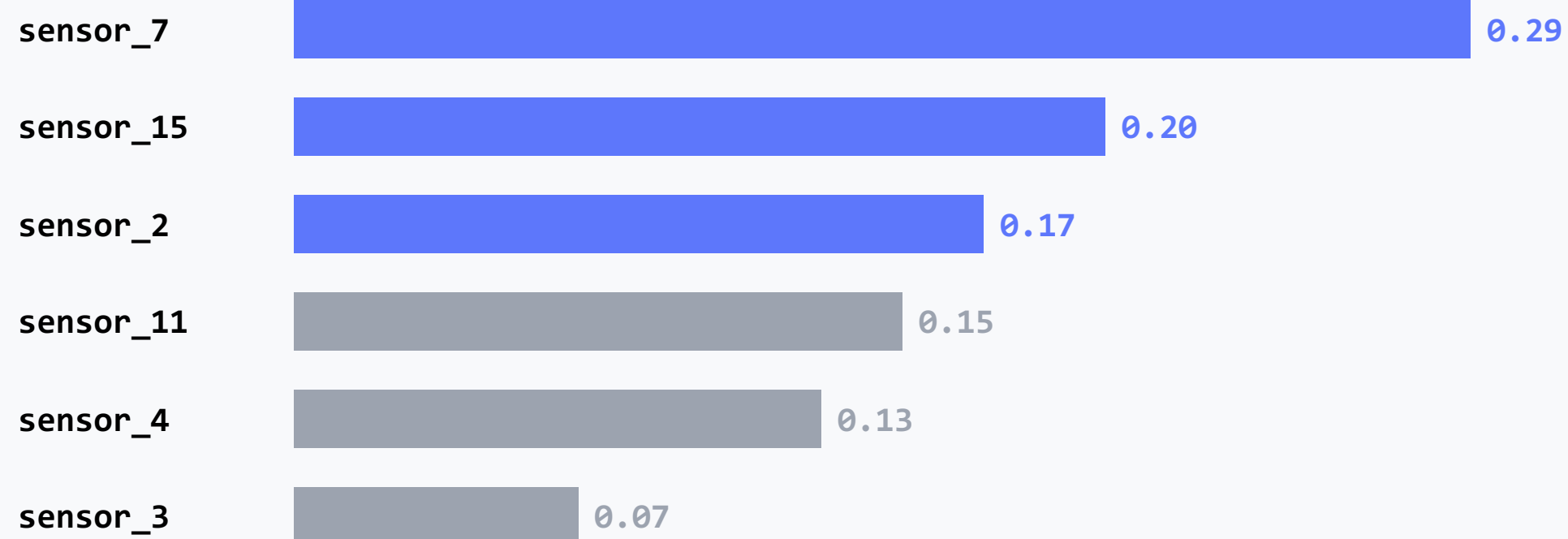
모든 점수의
합 = 1

각 센서가 차지하는 비중처럼 읽을 수 있음

센서별 중요도 살펴보기

sensor_7과 sensor_15가 판단에 가장 크게 기여

FEATURE IMPORTANCES



점수 큰 순으로 정렬 — 점수의 합은 1

READING

sensor_7 · sensor_15가 두드러진 두 센서

- 고장 판단에 가장 큰 역할
- 이 부위를 정비팀이 우선 점검
- 이 값은 데이터에 따라 달라짐

중요한 센서가 측정하는 부위를 정비팀이 먼저 살펴보면 좋다는 힌트

중요도는 원인이 아니라 단서

판단에 많이 쓰였다 ≠ 고장의 원인 — 힌트로 활용		
관점	오해	올바른 해석
중요도가 높다	그 센서가 고장의 원인	판단에 많이 쓰였다는 뜻일 뿐
활용 방법	단독으로 원인 단정	힌트로 사용 + 전문가 검증
중요도의 가치	블랙박스 모델	판단 근거를 사람이 점점 가능
고장 직전에 함께 변하는 좋은 단서 — 진짜 원인은 전문가가 확인		

예측 결과를 정비 관점으로 읽기

모델의 0/1을 현장의 말로 번역 — 1 + 높은 확신 = 가장 시급

예측 1

곧 고장 —
점검 필요 설비

위험 신호 — 빠른 점검이 손실을 막을 가능성 큼

예측 0

정상 운전
이어가도 됨

당장의 점검 대상은 아님 — 단, 주기 점검은 계속

확신도 결합

1 + 높은 확신
= 가장 시급

1이지만 애매하면 한 번 더 지켜볼 대상으로 분류

중요 센서까지 함께 전달

"어디를 왜 살펴야 하는지"까지 전달 — 분석이 현장 행동으로

STEP 1

곧 고장 확률 높은 설비 선정

1로 예측 + 확신도 높음 = 우선 안내 대상

예측 + 확신도 결합



STEP 2

중요 센서 부위 점검 안내

feature importance에서 얻은 정보로 어디를 먼저 보라고 전달

어디를 · 왜 살펴야 하는지



STEP 3

현장 행동으로 연결

데이터 분석 결과가 정비팀의 구체적 행동으로 이어짐

예지보전의 마침표

예측을 점검 우선순위로 바꾸기

1로 예측된 설비를 확률 높은 순으로 정렬 → 위에서부터 점검

01 · 필터링

1로 예측된 설비를 모아
우선 점검 후보

예측값 == 1 필터로 위험 후보군 추출

02 · 정렬

곧 고장 확률
높은 순으로 줄 세우기

predict_proba의 1일 확률로 내림차순 정렬

03 · 실행

위에서부터 점검
가장 위험한 설비부터

한정된 인력·시간을 가장 시급한 곳부터 차례로 배치

한정된 자원을 효과적으로

인력·시간을 가장 위험한 곳에 집중 — 예지보전이 작동하는 모습

01 · 자원 배분

위험한 곳에
먼저 집중

인력과 시간이 부족할 때 우선순위가 분명한
결정 가능

02 · 두 종류의 낭비

고장 손실 + 헛점검 낭비
함께 감소

갑작스러운 고장 손실과 불필요한 점검을 동
시에 줄임

03 · 의미

예지보전이 실제로
작동하는 모습

예측 → 우선순위 → 점검의 한 사이클이 손에
잡힘

MIMII로 같은 분류 흐름 반복하기

히타치 공개 설비 작동음 데이터셋 — 소리로 상태를 판단

01 · 데이터셋

MIMII

히타치 공개 데이터셋

공장 설비의 작동 소리를 녹음해 공개한 음향 데이터

02 · 담긴 소리

밸브·펌프·팬의
정상음과 이상음

설비 종류별로 정상 소리와 이상 소리를 함께
담음

03 · 판단 매체

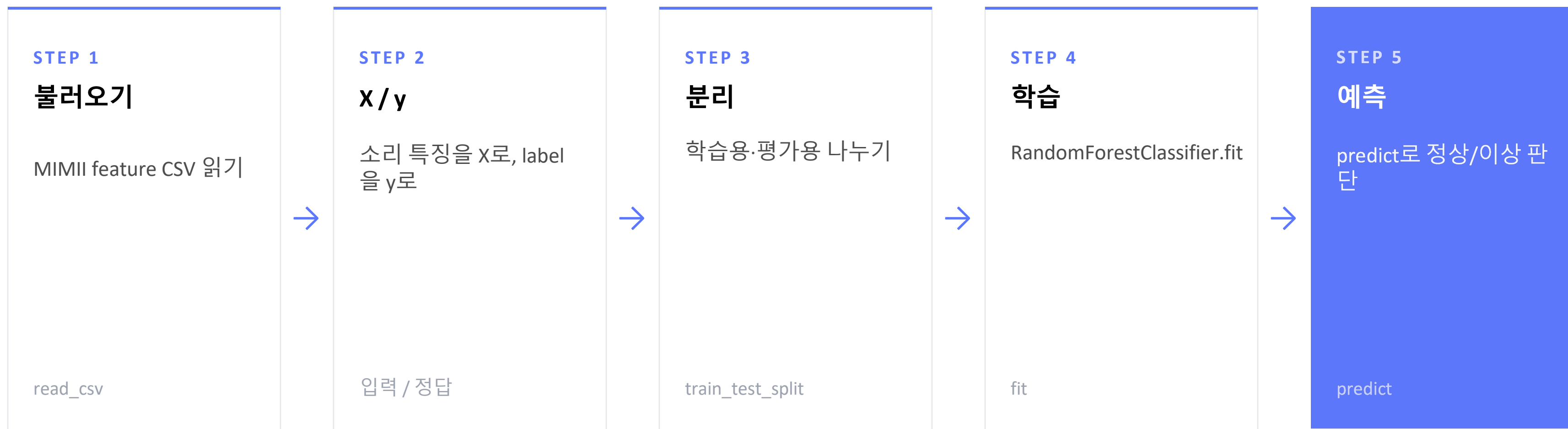
센서 숫자가 아닌
소리로 상태 판단

마이크도 일종의 센서 — 소리에서 특징값을
뽑아 입력으로 사용

같은 흐름을 전혀 다른 데이터(소리)에 적용해 일반성 확인

데이터는 달라도 문제는 같다

정상(0)/이상(1) 이진 분류 — 데이터만 갈아 끼우면 됨



엔진 때 익힌 흐름을 글자 하나 바꾸지 않고 그대로 적용 — 데이터만 갈아 끼우면 됨

MIMII feature CSV의 구조

소리를 숫자 특징으로 요약한 feature CSV를 그대로 사용

컬럼	의미
rms	소리의 평균적인 크기
spectral_centroid	소리 에너지의 무게중심 주파수
zero_crossing_rate	파형이 기준선을 넘나드는 빈도
label	정상 0 / 이상 1

각 행 = 소리 한 토막 · 특징 숫자 컬럼들 + 정답 label · C-MAPSS와 똑같은 방식

C-MAPSS와 똑 닮은 구조

입력 여러 개 + 정답 하나 — 표의 골격이 같으면 같은 방식으로 다룸

C-MAPSS · 엔진

센서 컬럼 = 입력 x
failure_soon = 정답 y

- sensor_2, sensor_3, sensor_4 ...
- RUL에서 만든 0/1 라벨
- 행 = 한 시점의 운전 상태

MIMII · 소리

소리 특징 = 입력 x
label = 정답 y

- rms, spectral_centroid ...
- 0(정상) / 1(이상) 라벨
- 행 = 한 토막의 소리

소리 특징으로 정상/이상을 나눈다는 것

정상 소리와 이상 소리의 특징값이 다른 영역에 모이는 경계를 학습

01 · 변화

고장이 다가오면 소리의
크기·높낮이·거칠기 변화

베어링이 닳으면 rms 상승, 날카로운 소리 섞이면 무게중심 변화

02 · 분포 차이

정상/이상의 특징값이
다른 영역에 모임

정상 소리와 이상 소리의 숫자 패턴이 서로 다른 곳에 무리지어 있음

03 · 모델의 역할

그 경계를 학습해
둘을 가름

데이터에서 자동으로 경계를 익혀 새 소리를 분류

청진기 비유

의사가 청진으로 정상 범위를 익히듯 모델은 데이터로 그 범위를 학습

01 · 의사

청진기로 심장 소리를 듣고
정상/이상 판단

경험으로 정상 소리의 범위를 익혀둔 상태

02 · 모델

데이터로
정상 범위를 학습

의사의 경험 대신 학습 데이터로 정상 패턴 익힘

03 · 판단

새 소리가 정상 범위를
벗어나면 이상

의미를 깊이 몰라도 값이 다른 영역에 있으면
경계 학습 가능

전체 파이프라인 — 데이터 준비

앞쪽 절반 — 데이터를 모델이 먹을 형태로 다듬는 과정

STEP 1

불러오기

데이터 읽기 — read_csv로 CSV를 표로 가져옴

pandas DataFrame



STEP 2

라벨

RUL로 failure_soon 만들기

$RUL \leq 30 \rightarrow 1$



STEP 3

x / y 나누기

센서를 x, 정답을 y로 분리

입력 / 정답

이 세 단계로 모델에게 보여줄 표가 준비됨 — 다음은 모델 다루기

전체 파이프라인 — 학습과 해석

뒤쪽 절반 — 모델로 학습·예측하고 결과를 해석하는 과정

STEP 4

분리

학습용·평가용으로 나누기

`train_test_split`



STEP 5

학습·예측

fit으로 학습, predict로 예측

`RandomForestClassifier`



STEP 6

해석

비교·중요도·우선순위

데이터 → 현장 판단

이 흐름의 가장 큰 가치 — 데이터가 바뀌어도 그대로 통한다

이 파이프라인은 재사용된다

새 분류 문제를 만나면 이 여섯 단계를 차례로 떠올리기

01 · 흐름

여섯 단계를
차례로 떠올리기

새 문제 앞에서 막막할 때 이 여섯 칸을 채워 나가면 실마리가 보임

02 · 세부 코드

그때그때
찾아 쓰면 됨

함수 이름이나 옵션은 문서·검색으로 채우면
충분

03 · 가장 값진 자산

전체 흐름을
잃지 않는 것

이 흐름을 스스로 끝까지 밟아 본 경험이 가장
값진 자산

우리가 만든 예측기의 의미와 한계

데이터로 위험을 미리 짚는 예지보전을 직접 구현 — 단, 보조 수단

01 · 의미

예지보전을
직접 구현

센서값만 보고 곧 고장날 설비를 처음부터 끝
까지 골라내는 도구

02 · 한계

완벽한 정답기가 아닌
근거 있는 추정

본 적 없는 데이터엔 약하고 데이터 품질에 좌
우됨

03 · 활용

사람의 판단을 돕는
보조 수단

예측이 얼마나 정확한지 숫자로 재는 방법은
다음 파트

04

실습

랜덤 포레스트로 학습·예측·해석·MIMII 확장·종합

실습 — 모델 만들고 학습시키기

모듈 2에서 만든 `x_train, y_train`으로 랜덤 포레스트 학습

GOAL 01

RandomForestClassifier
생성

`n_estimators=100, random_state=42`로 시작

GOAL 02

학습 데이터로
`.fit()` 실행

`X_train, y_train` 두 인자만 전달

실습 — 분류기 불러오기

sklearn.ensemble에서 RandomForestClassifier импорт

CODE · IMPORT.PY

```
from sklearn.ensemble import RandomForestClassifier
```

실습 — 모델 생성

트리 100그루 모델 생성 — 아직 학습 전 빈 상자

CODE · CREATE.PY

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

실습 — 학습 실행

학습용 데이터로 학습 — 평가용은 절대 넣지 않음

CODE · FIT.PY

```
model.fit(X_train, y_train)
```

CHECKPOINT

오류 없이 실행됐는가? 평가용(test)은 넣지 않았는가?

학습용 X_train, y_train 두 인자만 fit에 들어갔는지 확인

실습 — 학습 확인

모델 변수를 출력해 학습이 끝났는지 확인

CODE · CHECK.PY

```
model
```

실습 — 예측하고 결과 형태 확인

평가용 데이터로 예측 받고 결과가 0/1 배열임을 확인

GOAL 01

평가용 데이터로
예측 받기

아껴 둔 `x_test`로 `predict` 호출

GOAL 02

결과가 0/1 배열인지
개수가 맞는지 점검

예측 개수 = 평가용 행 수

실습 — 예측 실행

예측 결과를 y_pred에 담고 출력 — 0/1로만 이루어진 배열인가?

CODE · PREDICT.PY

```
y_pred = model.predict(X_test)
y_pred
```

실습 — 개수 확인

예측 개수와 평가용 행 개수가 같은지 확인

CODE · COUNT.PY

```
len(y_pred)  
X_test.shape
```

실습 — 앞부분 보기

예측의 앞 10개만 잘라 보기 — 순서는 x_test와 동일

CODE · HEAD.PY

```
y_pred[:10]
```

실습 — 결과 형태 정리

예측 = 행마다 0/1이 담긴 배열, 입력과 같은 순서

NOTE 01

행마다 0/1이 담긴 배열
순서는 **입력과 동일**

입력 100행이면 답도 100개, `x_test`의 행 순서를 그대로 유지

NOTE 02

이 순서 덕분에
실제 정답과 나란히 비교

예측·실제·설비 번호 모두 같은 순서로 맞춰 볼 수 있음

실습 — predict_proba로 확신도 살펴보기

각 행의 0일 확률과 1일 확률 확인 — 강한 확신/애매한 행 비교

GOAL 01

각 행의
0일 확률 · 1일 확률 확인

predict_proba가 돌려주는 두 열의 의미 파악

GOAL 02

확신이 강한 행과
애매한 행을 비교

0.9 vs 0.55 같은 행을 직접 들여다보기

실습 — 확률 받기

예측 확률을 받아 앞 5개 확인 — 두 확률을 더하면 1이 되는가?

CODE · PROBA.PY

```
proba = model.predict_proba(X_test)
proba[:5]
```

실습 — 1일 확률만 보기

두 번째 열, 즉 1일 확률(곧 고장 확률)만 따로 보기

CODE · PROBA_COL.PY

```
proba[:5, 1]
```

실습 — 확신도 해석

확률이 높은 행과 0.5 부근인 행을 비교해 확신도 차이를 느껴 보기

STRONG

**1일 확률 0.9 이상
강하게 확신**

100그루 중 90그루 이상이 1로 투표한 결과 — 즉시 점검 후보

WEAK

**0.5 부근
확신이 약함**

트리들 의견이 갈린 경우 — 다음 사이클 관찰 대상

실습 — random_state 바꿔 다시 학습

값을 바꾸면 결과가 조금 달라짐, 같은 값이면 똑같이 재현됨

실험 01

**random_state를 바꾸면
결과가 조금 달라질 수 있음**

트리들이 보는 데이터·센서 일부가 달라져서

실험 02

**같은 값이면 결과가
똑같이 재현됨**

수업·발표에서 동일한 결과를 공유할 수 있음

실습 — 다른 random_state로 학습

random_state를 7로 바꿔 새 모델을 학습하고 예측

CODE · MODEL2.PY

```
model2 = RandomForestClassifier(n_estimators=100, random_state=7)
model2.fit(X_train, y_train)
y_pred2 = model2.predict(X_test)
```

실습 — 예측 비교

앞 예측과 새 예측의 앞부분 비교 — 일부 예측이 조금 달라졌는가?

CODE · COMPARE.PY

```
y_pred[:10]  
y_pred2[:10]
```

실습 — 같은 값이면 재현

같은 random_state 42로 다시 학습해 결과가 똑같은지 확인

CODE · REPRODUCE.PY

```
model3 = RandomForestClassifier(n_estimators=100, random_state=42)
model3.fit(X_train, y_train)
(model3.predict(X_test) == y_pred).all()
```

실습 — 예측 비교 + feature importance

예측·실제 눈 비교 + 어떤 센서가 판단에 크게 쓰였는지 확인

GOAL 01

예측과 실제를
나란히 비교

점수 계산은 다음 파트 — 지금은 눈으로 감만 잡기

GOAL 02

feature_importances_로
중요 센서 확인

판단 근거가 된 센서를 점수로 확인

실습 — 예측·실제 나란히 보기

예측과 실제를 한 표에 담아 앞부분 확인

CODE · COMPARE_DF.PY

```
import pandas as pd
compare = pd.DataFrame({'예측': y_pred, '실제': y_test.values})
compare.head(15)
```

실습 — 눈으로 일치 세기

표를 보며 예측과 실제가 같은 행이 많은지 확인

NOTE 01

표를 눈으로 훑어
대체로 잘 맞는지 감 잡기

같은 값끼리 짝지어진 행이 더 많은지 살펴봄

NOTE 02

정확한 점수는
다음 파트의 평가 지표

accuracy · precision · recall 등 숫자로 정밀 측정

실습 — 중요 센서 확인

각 센서의 중요도를 큰 순으로 정렬

CODE · IMPORTANCE.PY

```
importance = pd.Series(model.feature_importances_, index=features)
importance.sort_values(ascending=False)
```

실습 — 중요도 활용

중요한 센서 부위를 우선 점검 안내 — 모델 판단이 현장 행동으로

POINT 01

중요도 높은 센서 부위를
우선 점검 안내

sensor_7·sensor_15가 측정하는 부위를 정비팀이 먼저 살펴봄

POINT 02

모델 판단을
현장 행동으로 연결

분석 결과가 구체적 행동 지시로 이어짐

실습 — 예측 결과를 표로 모아 비교

예측·실제·곧고장확률을 한 표에 모아 정비 우선순위로 가공

GOAL 01

예측·실제·곧고장확률을
한 표로 모으기

DataFrame 세 컬럼 — 한눈에 모델 출력 전부 확인

GOAL 02

곧 고장 예측 설비를 추려
우선순위 만들기

위험한 순으로 정렬해 정비팀에 그대로 전달 가능

실습 — 결과 표 만들기

예측·실제·곧고장확률을 한 표에 — 확률은 1일 확률 사용

CODE · RESULT_DF.PY

```
result = pd.DataFrame({'예측': y_pred, '실제': y_test.values, '곧고장확률': proba[:, 1]})  
result.head(10)
```

실습 — 곧 고장만 추리기

1로 예측된 행만 추리기 — 우선 점검 후보

CODE · FILTER.PY

```
result[result['예측'] == 1].head(10)
```

실습 — 우선순위 정렬

곧 고장 확률 높은 순으로 정렬해 우선순위 만들기

CODE · SORT.PY

```
result.sort_values('곧고장확률', ascending=False).head(10)
```

실습 — 표 활용 정리

정렬된 표 자체가 정비 우선순위 목록 — 그대로 정비팀 전달 가능

USE 01

정렬된 위쪽 설비부터
점검

가장 위험한 곳부터 손볼 수 있는 자연스러운 순서

USE 02

이 표 그대로
정비팀에 전달 가능

DataFrame 자체가 정비팀에게 줄 수 있는 결과물

실습 — MIMII feature CSV로 정상/이상 분류

소리 특징 데이터에 똑같은 분류 흐름 적용 — 데이터만 바뀔 뿐

GOAL

소리 특징 데이터에
똑같은 분류 흐름 적용

불러오기·x/y·분리·학습·예측 단계는 엔진 때와 동일

DATA

mimii_features
_sample.csv

rms · spectral_centroid · zero_crossing_rate · label

실습 — 소리 데이터 불러오기

rms, spectral_centroid, zero_crossing_rate, label 컬럼 확인

CODE · MIMII_LOAD.PY

```
mdf = pd.read_csv('mimii_features_sample.csv')  
mdf.head()
```

실습 — x/y 나누기

소리 특징 세 개를 입력 mX 로, label을 정답 my 로 분리

CODE · MIMII_XY.PY

```
mX = mdf[['rms', 'spectral_centroid', 'zero_crossing_rate']]  
my = mdf['label']
```

실습 — 학습용/평가용 분리

엔진 때와 똑같이 분리 — stratify로 정상/이상 비율 유지

CODE · MIMII_SPLIT.PY

```
from sklearn.model_selection import train_test_split
mX_train, mX_test, my_train, my_test = train_test_split(
    mX, my, test_size=0.2, random_state=42, stratify=my)
```

실습 — 학습과 예측

랜덤 포레스트로 학습하고 예측 — 코드 흐름이 엔진 때와 거의 동일

CODE · MIMII_FIT.PY

```
mmodel = RandomForestClassifier(n_estimators=100, random_state=42)
mmodel.fit(mX_train, my_train)
my_pred = mmodel.predict(mX_test)
my_pred[:10]
```

실습 — 소리 분류 정리

데이터만 바뀌었을 뿐 흐름은 동일 — 분류의 본질은 같음

NOTE 01

데이터만 바뀌었을 뿐
흐름은 동일

엔진 센서값 → 소리 특징, 골격은 그대로

NOTE 02

같은 분류 틀이
소리 데이터에도 통함

진동·온도·소리 — 측정 도구만 다를 뿐, 분류 본질은 동일

실습 — 종합 미니 실습 (전체 파이프라인)

불러오기 → 라벨 → x/y → 분리 → 학습 → 예측 전 과정을 한 흐름으로

GOAL 01

처음부터 끝까지
스스로 **이어서** 완성

나뉜 배운 모든 단계를 한 흐름으로 이어 붙여 예측기 완성

GOAL 02

막히면
앞 실습을 참고

헛갈리는 부분은 앞 슬라이드로 돌아가 확인

실습 — 불러오기와 라벨

데이터를 불러오고 RUL로 failure_soon 라벨 만들기

CODE · LOAD_AND_LABEL.PY

```
df = pd.read_csv('cmapss_fd001_sample.csv')
df['failure_soon'] = (df['RUL'] <= 30).astype(int)
```

실습 — X/y 구성

센서 여섯 개를 X로, failure_soon을 Y로 — RUL이 X에 없는지 확인

CODE · XY.PY

```
features = ['sensor_2', 'sensor_3', 'sensor_4', 'sensor_7', 'sensor_11', 'sensor_15']
X = df[features]
y = df['failure_soon']
```

실습 — 데이터 분리

학습용과 평가용으로 나누기 — stratify=y로 비율 유지

CODE · SPLIT.PY

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

실습 — 학습과 예측

랜덤 포레스트로 학습하고 예측 — 여섯 단계를 막힘 없이 이었나?

CODE · FIT_PREDICT.PY

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

실습 — 종합 완성 점검

여섯 단계를 스스로 완성 — 오류가 나면 어느 단계인지 찾아 고치기

CHECK 01

여섯 단계를
순서대로 완성

불러오기·라벨·x/y·분리·학습·예측 — 모두 막힘 없이 이어졌는지

CHECK 02

오류가 나면
어느 단계인지 찾아 고치기

오류 위치를 짚어 고치는 것도 중요한 능력

실습 — 종합 결과 읽고 정리

예측·실제·확신도·중요 센서를 정비 우선순위로 해석하며 마무리

GOAL

설비별로 결과 정리,
현장 전달 **형태**로 해석

예측·실제·확신도를 한 표에 모아 우선순위 목록 완성

MEANING

데이터에서 시작해
현장 판단으로 끝나는 한 바퀴

분석에서 행동까지 — 예지보전의 완성된 한 흐름

실습 — 결과 표와 눈비교

예측·실제·곧고장확률을 모아 앞부분 눈비교 — 점수는 다음 파트

CODE · RESULT_VIEW.PY

```
proba = model.predict_proba(X_test)
result = pd.DataFrame({'예측': y_pred, '실제': y_test.values, '곧고장확률': proba[:, 1]})
result.head(15)
```

실습 — 우선순위와 중요 센서

곧 고장 확률 순 우선 점검 목록 + 어떤 센서가 크게 쓰였는지 확인

CODE · PRIORITY.PY

```
result.sort_values('곧고장확률', ascending=False).head(10)  
pd.Series(model.feature_importances_, index=features).sort_values(ascending=False)
```

실습 — 한 문장으로 정리

곧 고장 확률 높은 설비부터 점검, 중요 센서 부위 우선 살피기, 사람이 검토

01 · 점검 순서

곧 고장 확률 높은
상위 설비부터 점검

한정된 자원을 가장 위험한 곳에 먼저 배치

02 · 점검 위치

가장 중요한 센서
부위를 우선 살핌

feature importance가 알려주는 단서를 정비 행동으로 전환

03 · 자세

예측은 사람이
검토하는 출발점

모델은 보조 수단 — 최종 판단은 사람이